

تسک ارزیابی توسعه دهنده بک اند (ASP.NET Core 8)

سیماگر - سامانه مدیریت هوشمند فروشگاه

نسخه آماده انتشار برای آگهی استخدام و ارسال به داوطلبان

8) ASP.NET Core) تسک ارزیابی توسعه‌دهنده بک‌اند

سیماگر - سامانه مدیریت هوشمند فروشگاه

۱. هدف کلی

CQRS، Vertical Slices، Clean Architecture و Domain-Driven Design طراحی و پیاده‌سازی ماژول مدیریت سبد خرید مطابق با معماری، که در پروژه‌ی جاری تعریف شده است. این تسک Architecture و Domain-Driven Design توانایی داوطلب را در پیاده‌سازی الگوهای پیشرفته، رعایت استانداردهای کدنویسی و یکپارچه‌سازی می‌سنجد و RabbitMQ و Redis با ابزارهای جانبی مانند

۲. سناریوی کسب‌وکار

فروشگاه اینترنتی «سیماگر» نیازمند سیستمی برای مدیریت سبد خرید کاربران است. هر کاربر پس از ورود به سامانه، می‌تواند

- سبد خرید خود را مشاهده کند (در صورت نبودن، سبد جدید ساخته شود)
- یک یا چند کالا با شناسه و تعداد مشخص به سبد اضافه کند
- تعداد یک کالای موجود در سبد را تغییر دهد
- یک کالا را از سبد حذف کند
- تمام کالاهای سبد را یکجا خالی کند
- پس از ۳۰ دقیقه بی‌حرکتی کاربر، سبد خرید به‌طور خودکار منقضی شود (با استفاده از RabbitMQ) رویداد دامنه و

۳. ساختار پوشه‌های پروژه (مطابق با مشخصات)

کدهای تولیدی باید دقیقاً در پوشه‌های زیر قرار گیرند

```
src/
├── ProjectName.API/
│   └── Controllers/
│       └── BasketController.cs
├── ProjectName.Application/
│   └── Basket/
│       ├── Commands/
│       │   ├── AddItemToBasketCommand.cs
│       │   ├── AddItemToBasketCommandHandler.cs
│       │   ├── UpdateBasketItemQuantityCommand.cs
│       │   ├── UpdateBasketItemQuantityCommandHandler.cs
│       │   ├── RemoveBasketItemCommand.cs
│       │   ├── RemoveBasketItemCommandHandler.cs
│       │   ├── ClearBasketCommand.cs
│       │   ├── ClearBasketCommandHandler.cs
│       │   └── ExpireBasketsCommand.cs (و Handler آن)
│       ├── Queries/
│       │   ├── GetOrCreateBasketQuery.cs
│       │   └── GetOrCreateBasketQueryHandler.cs
│       ├── DTOs/
│       │   ├── BasketDTO.cs
│       │   ├── BasketItemDTO.cs
│       │   └── AddBasketItemDTO.cs
│       └── Events/
│           ├── BasketItemAddedEvent.cs
│           └── BasketItemAddedEventHandler.cs (اختیاری)
├── ProjectName.Domain/
│   ├── Entities/
│   │   ├── Basket.cs
│   │   └── BasketItem.cs
│   ├── Enums/
│   │   └── BasketStatus.cs
│   └── Events/
│       └── BasketExpiredEvent.cs
└── ProjectName.Infrastructure/
    ├── Persistence/
    │   ├── Configurations/
    │   │   └── BasketConfiguration.cs
    │   └── Repositories/
    │       └── BasketRepository.cs (و IBasketRepository)
    └── Services/
        ├── Redis/
        │   └── IBasketCacheService.cs (و پیاده سازی)
        └── MessageBus/
            └── IBasketEventPublisher.cs (و پیاده سازی)
```

شرح دقیق تسک‌ها ۴.

۴-۱. مدل‌های دامنه (Domain Layer)

Basket الف (کلاس)

ویژگی	نوع	توضیح
Id	long	شناسه یکتا
UserId	long	شناسه کاربر
Status	BasketStatus	(Active یا Expired) وضعیت
Items	List<BasketItem>	لیست آیتم‌های سبد
CreatedAt	DateTime	زمان ایجاد
LastUpdatedAt	DateTime?	آخرین بروزرسانی

قوانین دامنه:

- حداکثر تعداد هر کالا در سبد: ۱۰ عدد
- نباید از ۵۰,۰۰۰,۰۰۰ ریال بیشتر (Sum of (Quantity * UnitPrice)) مجموع قیمت سبد شود؛ در غیر این صورت عملیات افزودن/تغییر تعداد با خطا مواجه شود.

متدهای دامنه (با منطق کامل)

- AddItem(ProductId, Quantity, UnitPrice)
- UpdateQuantity(ProductId, NewQuantity, UnitPrice)
- RemoveItem(ProductId)
- Clear()
- BasketExpiredEvent و ایجاد رویداد Expired (تغییر وضعیت به) Expire()

BasketItem ب (کلاس)

ویژگی	نوع	توضیح
Id	long	شناسه یکتا
ProductId	long	شناسه کالا
Quantity	int	تعداد
UnitPrice	decimal	قیمت واحد در زمان افزودن

Enum BasketStatus ج)

Active = 1، Expired = 2: مقادیر

BasketExpiredEvent د (رویداد دامنه)

است. UserId و BasketId این رویداد هنگام انقضای سبد ایجاد شده و شامل

۴-۲. Command ها و Query ها (CQRS)

-
- Query ها و <TResponse> ICommand ها باید اینترفیس Command همه را پیاده‌سازی کنند <TResponse> IQuery
- الف) GetOrCreateBasketQuery
- ورودی: UserId (long)
 - خروجی: ServiceResult<BasketDTO>
 - رفتار: اگر سبد فعال برای کاربر وجود داشت، برگرداند؛ در غیر این صورت یک سبد جدید با بسازد و ذخیره کند Active وضعیت
- ب) AddItemToBasketCommand
- ورودی: UserId، AddBasketItemDTO (شامل ProductId و Quantity)
 - خروجی: ServiceResult
 - اعتبارسنجی (FluentValidation):
 - o ProductId > 0
 - o بین ۱ تا ۱۰ Quantity
 - را صادر کند (برای ثبت تاریخچه) BasketItemAddedEvent پس از موفقیت، رویداد
- ج) UpdateBasketItemQuantityCommand
- ورودی: UserId، ProductId، NewQuantity
 - خروجی: ServiceResult
 - بین ۱ تا ۱۰ NewQuantity: اعتبارسنجی
- د) RemoveBasketItemCommand
- ورودی: UserId، ProductId
 - خروجی: ServiceResult
- ه) ClearBasketCommand
- ورودی: UserId
 - خروجی: ServiceResult
- و) ExpireBasketsCommand
- ورودی: بدون پارامتر
 - خروجی: ServiceResult
 - هر ۱ (RabbitMQ یا BackgroundService) Consumer توسط یک Command این دقیقه اجرا می‌شود
 - آنها بیش از ۳۰ دقیقه قبل بوده و وضعیت LastUpdatedAt وظیفه: یافتن سبدهایی که را روی آنها فراخوانی کند () Expire دارند؛ سپس متد Active

۴-۳. Handlerها

- متناسب خود را داشته باشد Handler باید Command/Query هر
- برای دسترسی به داده استفاده IUnitOfWork و IRepository<Basket> ها از Handler کنند.
- تغییرات وضعیت سبد فقط از طریق متدهای دامنه صورت گیرد (نه تنظیم مستقیم خصوصیات).
- باشد <T> ServiceResult یا ServiceResult خروجی همیشه از نوع

۴-۴. Pipeline Behaviors

<ValidationBehavior<TRequest, TResponse> (الف)

- هستند، اعتبارسنجی را Validator هایی که دارای Command به طور خودکار برای تمام اجرا کند.
- و پیام خطا BadRequest با کد وضعیت ServiceResult، در صورت شکست اعتبارسنجی برگرداند.

<TransactionBehavior<TRequest, TResponse> (ب)

- تراکنش دیتابیس را آغاز کند (ها Query نه) ها Command برای تمام
- استخراج و با DbContext را از Domain Events، Handler پس از اجرای ارسال کند IDomainEventDispatcher
- کند Commit را فراخوانی و در نهایت تراکنش را SaveChangesAsync سپس
- شود Rollback در صورت بروز استثناء، تراکنش به طور خودکار

۴-۵. Repository و UnitOfWork

- (که از پیش تعریف شده اند) Repository<Basket> و IRepository<Basket> از شود.
- ارث‌بری کرده IRepository<Basket> که از IBasketRepository یک اینترفیس سفارشی را اضافه کند (GetActiveBasketByUserIdAsync(userId, cancellationToken) و متد
- ارث‌بری Repository<Basket> که از BasketRepository پیاده‌سازی این متد در کلاس می‌کند (انجام شود)

۴-۶. کنترلر API (BasketController)

مسیر پایه: api/v1/basket

HTTP متد مسیر توضیح

GET {userId} دریافت یا ایجاد سبد

POST {userId}/items افزودن کالا

PUT {userId}/items/{productId} تغییر تعداد کالا

DELETE {userId}/items/{productId} حذف کالا

DELETE {userId} خالی کردن کل سبد

- تبدیل شوند ApiResponse به فرمت () ToApiResponse تمام پاسخ‌ها باید با متد توسعه‌یافته
- و ICommandDispatcher ارث‌بری کرده و از BaseController کنترلر باید از IQueryDispatcher استفاده کند

RabbitMQ رویدادها و یکپارچه‌سازی با ۴-۷.

BasketExpiredEventHandler (الف)

- را پیاده‌سازی کند <BasketExpiredEvent> IDomainEventHandler این کلاس
- لاگ ثبت کند و (اختیاری) ایمیلی به کاربر ارسال نماید، HandleAsync در متد ExpireBasketsCommand برای Consumer (ب)
- ایجاد شود که هر ۱ RabbitMQ واقعی Consumer یا یک BackgroundService یک را ارسال کند ExpireBasketsCommand دقیقه یک
- پردازش شده و سبدهای ExpireBasketsCommandHandler توسط Command این منقضی را به‌روز کند

۴-۸. کش (Redis)

GetOrCreateBasketQueryHandler الف (در)

- basket:{UserId} کلید کش
- برگردانده شود BasketDTO دریافت شود؛ در صورت وجود، به‌عنوان Redis ابتدا از
- با زمان انقضای ۵ دقیقه ذخیره Redis در صورت عدم وجود، از دیتابیس خوانده شده، در شود.
- کردن کش Invalidate (ب)
- پس از هر تغییری در سبد (افزودن، حذف، تغییر تعداد، خالی کردن یا انقضا)، کلید حذف شود Redis مربوطه از

تست‌نویسی (امتیازی) ۴-۹.

- و Moq با استفاده از AddItemToBasketCommandHandler نوشتن تست واحد برای

xUnit.

- و WebApplicationFactory نوشتن تست یکپارچه برای کنترلر با استفاده از (اختیاری اما تشویق می‌شود) SQL Server برای TestContainers.

۵. الزامات تحویل

۱. (یا آرشیو فشرده) با رعایت ساختار پوشه‌های مشخص شده Pull Request کد کامل در قالب یک
۲. شامل README فایل
- (شامل SQL Server، Redis، RabbitMQ) Docker Compose نحوه اجرا با
- توضیح تصمیمات معماری و چالش‌های پیش‌رو
- (Postman یا Swagger با) API اسکرین‌شات از خروجی
۳. EF Core Migration با استفاده از BasketItem و Basket برای ایجاد جداول Migration فایل
۴. برای ماژول C4 Model یا Sequence Diagram (اختیاری) مستندات

۶. زمان پیشنهادی

حداکثر ۸ ساعت کاری (امکان انجام در چند روز)

۷. معیارهای ارزیابی

معیار امتیاز

- ۲۰ رعایت معماری (CQRS، DDD، Vertical Slices)
- ۱۵ کیفیت کد و رعایت استانداردها (نام‌گذاری، کامنت فارسی، ساختار)
- ۱۵ Domain Events و UnitOfWork، Repository پیاده‌سازی
- ۱۰ Pipeline Behaviors اعتبارسنجی و
- ۱۰ (RabbitMQ) و پیام‌رسان (Redis) مدیریت کش
- ۱۰ API Result کنترلر و
- ۱۰ مدیریت خطا و تراکنش
- ۱۰ تست‌نویسی (در صورت انجام)

نکات مهم برای داوطلب ۸.

- در جای مناسب استفاده Expression-bodied Members و Primary Constructors از کنید.
- تمام کامنت‌ها را به فارسی بنویسید.
- ، که در پروژه تعریف شده <TResponse> IQuery و <TResponse> ICommand حتماً از استفاده کنید.
- ، IScopedDependency برای ثبت وابستگی‌ها، از مارکر اینترفیس‌های ITransientDependency و ISingletonDependency بهره ببرید.
- برای شبیه‌سازی BackgroundService آشنا نیستید، می‌توانید از RabbitMQ اگر با جایگزین RabbitMQ توضیح دهید که در محیط تولید با README مصرف‌کننده استفاده کنید و در خواهد شد.
- استفاده از ابزارهای هوش مصنوعی در صورت ارائه پرامپت و توضیح خط فکری، بلامانع است.

نحوه ارسال پاسخ ۹.

(یا آرشیو زیپ (ارسال کرده و GitHub عمومی در Repository داوطلب باید نتیجه را به صورت یک لینک آن را در ایمیل درج نماید .همچنین یک ویدیوی کوتاه (حداکثر ۵ دقیقه (که در آن کد را ها را تست کند، امتیاز اضافی دارد API توضیح داده و

کلام پایانی ۱۰.

ما در سیمافر به دنبال افرادی هستیم که علاوه بر تسلط فنی، به معماری تمیز، مستندسازی و کیفیت کد اهمیت می‌دهند. این تسک فرصتی است تا توانایی‌های خود را در پیاده‌سازی یک ماژول واقعی و مقیاس‌پذیر نشان دهید. برای ما مهم است که کد شما خوانا، تست‌پذیر و مطابق با اصول باشد CQRS و DDD.

موفق باشید